LLM Beginner Roadmap

Five layers of vocabulary, from "what is a model" to MCP. Learn them bottom-up.

Every new term you hear belongs to one of these five layers, and each layer builds on the one before it. If a term confuses you, find its layer first, then read the layers below it.

1 What an LLM is

`token` `next-token prediction` `model` `training` `weights` `parameters` `transformer`

Token
A chunk of text, often part of a word. Models read and write tokens, not words.

Weights / parameters
The billions of numbers the model learns in training. They stay frozen while you use it.

> An LLM is trained on one task: predict the next token. All its other abilities come from doing that task at enormous scale.

→ The 35 Year History of ChatGPT (Art of the Problem, video)
→ Tokenizer playground: see how text is split into tokens
→ Attention Is All You Need: the transformer paper (optional, advanced)

2 Talking to a model

`prompt` `context window` `system prompt` `in-context learning` `zero-/few-shot` `temperature` `hallucination`

Context window
Everything the model can see at once: your prompt, the chat so far, attached files.

In-context learning
Teaching the model a new trick by putting examples in the prompt, with no retraining.

Hallucination
A confident answer that is false.

> The prompt is the program. A bigger context isn't automatically better: models miss facts buried mid-input and degrade over long chats. Start a new chat when you switch topics.

→ Lost in the Middle (paper)
→ LLMs Get Lost in Multi-Turn Conversation (~39% drop)

3 Giving it knowledge

`embeddings` `RAG` `vector` `semantic search` `vector database` `fine-tuning`

Embedding
Text turned into a list of numbers (a vector). Similar meanings land close together.

RAG
Retrieval-Augmented Generation: search your documents, paste the relevant bits into the prompt, then ask.

Fine-tuning
Further training that changes the model's weights.

> Need the model to know your data? Try RAG first. Fine-tuning is for changing style or behavior, and it costs more.

→ Willison: embeddings & RAG sections

4 Giving it actions

`tool use` `agent` `function calling` `workflow` `ReAct loop`

Tool use / function calling
The model asks your code to run a function (search, send email) and reads the result.

Agent
A model in a loop: think → call a tool → observe → repeat until done.

Workflow
A fixed sequence of LLM steps that you design ahead of time.

> Predictable steps → workflow. Steps you can't plan ahead → agent. Start simple.

→ Agents vs Workflows (talk)
→ Building Effective Agents (Anthropic)

5 Connecting it all: MCP

`MCP` `server` `client` `host` `tools` `resources` `prompts`

MCP (Model Context Protocol)
A standard plug that connects tools and data to any AI app, like USB for models.

Server
Exposes tools, resources, and prompts (e.g. a Slack server).

Host / client
The AI app (Claude, an IDE) and the connector inside it that talks to servers.

> Layers 1–4 explain why MCP exists: the model needs context (layer 3) and actions (layer 4), and MCP standardizes how it gets both.

→ modelcontextprotocol.io (official docs)
→ The Future of MCP (David Soria Parra, talk)
→ Build an MCP server with an LLM

⭐ Best single on-ramp

Catching up on the weird world of LLMs by Simon Willison. One read covers nearly every term on this page: how LLMs work, tips, tools, RAG, embeddings, training, open models, prompt injection.

Keep following: simonwillison.net

How to study this

Keep a running glossary. For each new term, write a one-line definition in your own words and tag its layer (1–5).

Use the model to learn the model. Ask it to explain a term, then explain it back and have it check you.

Going deeper (later)

The papers behind modern LLMs: Transformer · InstructGPT (how chat models learn to follow instructions) · Chinchilla (scaling).